



MCP + Multi-Agent Patterns + Final AI Systems Roadmap



We began with prompts. We end with systems coordination.

Today: protocols, multi-agent patterns, and how market-grade agentic systems evolve.

Goal: leave with a clear mental map, not just another library name.

Final zoom-out

**Prompt → Structure → Retrieval
→ Agents → Evaluation →
Frameworks → Protocols**

A good AI engineer understands where each layer belongs.



The Course as a Systems Stack

Prompt Layer
instructions, roles, temperature

Retrieval Layer
PDFs, chunks, embeddings, RAG

Evaluation Layer
LLM-as-a-Judge + confidence signals

Protocol + Multi-Agent Layer
MCP + role coordination

Structured Output Layer
JSON + Pydantic contracts

Agent / Workflow Layer
planner, tools, state, execution

Framework Layer
LangChain + LangGraph packaging

Every week added one more layer.
The final class is about coordination at the top of the stack.



How Our Program Compares to Market Agentic Systems

Our current system already has

- planner
- retrieval
- ranking / decision logic
- structured output validation
- judge / evaluation layer
- graph-style orchestration

Market-grade systems add

- durable execution
- human approval / HITL
- observability + tracing
- retries + fallbacks
- secure tool interfaces
- permissions + audit controls

**The gap is not only “more intelligence.”
The gap is production hardening.**

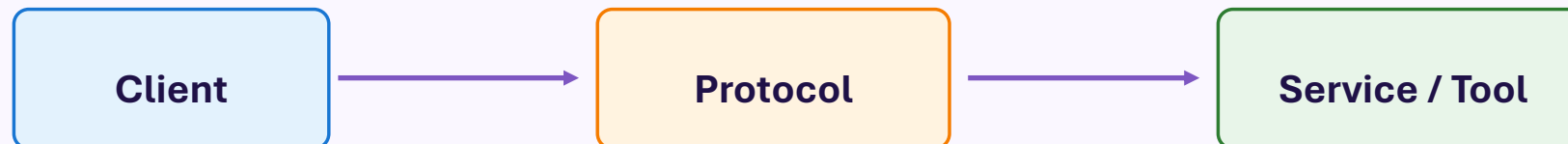
Before MCP: What Is a Protocol?

A protocol is an agreement about how parts of a system communicate.

It does not replace business logic — it standardizes interaction.

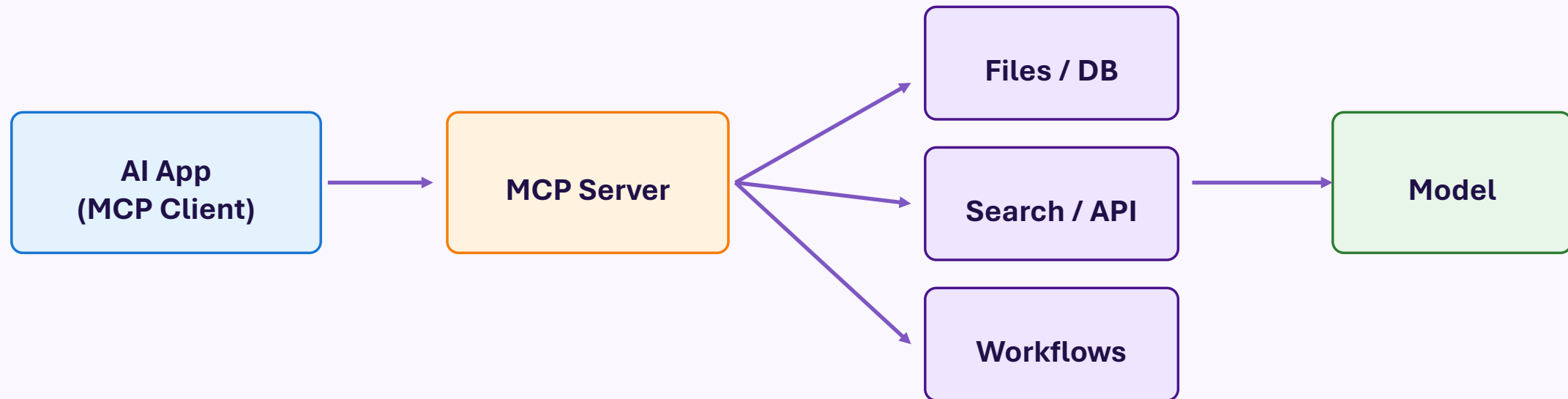
Examples students already know: HTTP, APIs, JSON contracts, database interfaces.

Protocol ≠ Intelligence
Protocol = Interface Agreement



MCP: Model Context Protocol

MCP is an open standard for connecting AI applications to external systems — data sources, tools, and workflows.



**MCP is not the engine.
It standardizes the ports and connectors.**



The Pain MCP Tries to Reduce

Without a standard

every tool has a custom interface
every app writes one-off glue code
tool access becomes hard to scale
security and permissions become scattered

With protocol-style thinking

tools expose consistent contracts
clients connect through shared patterns
capabilities become more portable
integration becomes easier to reason about

**MCP does not replace prompts, RAG, evaluation, or orchestration.
It helps standardize how external context and tools connect.**



Code Mindset: Tool Contracts Before MCP

```
tool_registry = {
    "resume_search": {
        "description": "Search resume evidence",
        "input_schema": {
            "query": "string",
            "top_k": "integer"
        },
        "output_schema": {
            "results": "list"
        }
    }
}
```

This is not a full MCP server.
But it teaches MCP-style thinking: clear tool names, clear inputs, clear outputs. Protocols are built on contracts.

This is the bridge from ad hoc Python tools to standardized tool/context interfaces.



Code Mindset: One Tool Entry Point

```
def call_tool(tool_name: str, payload: dict):  
    if tool_name == "resume_search":  
        query = payload["query"]  
        top_k = payload["top_k"]  
        return {  
            "results": retrieve_candidate_chunks(  
                query, vector_store, k=top_k  
            )  
        }  
  
    elif tool_name == "job_description_loader":  
        return {  
            "job_description": "We are hiring..."  
        }  
  
    else:  
        raise ValueError("Unknown tool")
```

Raw Python can already teach the interface idea.

One tool gateway receives structured payloads.

Every tool returns structured output.
MCP formalizes this kind of integration pattern.

**Same business logic.
Cleaner interaction surface.**



Protocol Power Comes with Responsibility

When models can access tools, security matters more.

Do not expose dangerous tools casually.

Add permissions, whitelists, audit logs, and human approval for sensitive actions.

Protocols make integration easier — they do not remove risk.

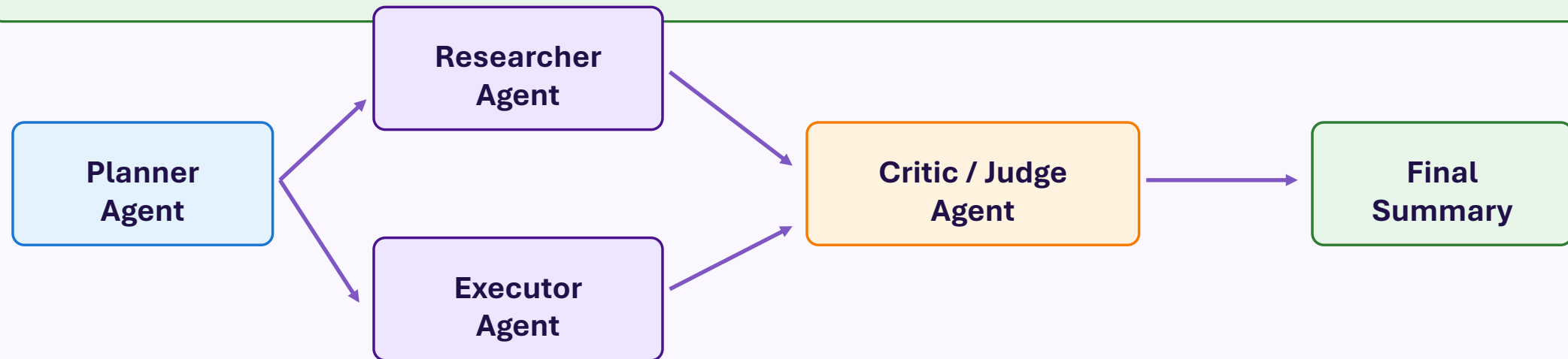
Safe tool design checklist

- **least privilege**
- **explicit inputs**
- **safe outputs**
- **confirmation gates**
- **logs + audit trail**

Market-grade agentic systems need controls, not just capability.

AutoGen + Multi-Agent Thinking

AutoGen-style systems organize multiple AI roles that can cooperate, critique, or coordinate to solve tasks.



**Multi-agent is not “more magic.”
It is role separation + coordination.**



Useful Multi-Agent Patterns

Planner + Executor
planner decomposes; executor acts

Generator + Critic
one creates; one critiques

Researcher + Summarizer
one gathers; one condenses

Specialists + Arbiter
roles debate; arbiter decides

Judge-on-Uncertainty
only invoke reviewer for close cases

Human-in-the-Loop
AI proposes; human approves

**More agents does not automatically mean a better system.
Multi-agent is a coordination strategy.**



Code Sketch: Role-Based Agents Without Framework Magic

```
def planner_agent(user_request: str):
    return {
        "task": "Find top resume candidates",
        "subtasks": [
            "Extract role requirements",
            "Retrieve resume evidence",
            "Summarize candidate fit",
            "Evaluate shortlist quality"
        ]
    }

def researcher_agent(plan: dict):
    return {"evidence": [
        {"candidate_name": "Ananya Patel", "score": 0.57},
        {"candidate_name": "Michael Chen", "score": 0.49}
    ]}

def judge_agent(shortlist: dict):
    return {
        "verdict": "Mostly grounded",
        "concerns": ["Michael lacks Selenium evidence"]
    }
```

This is not full AutoGen code.
It shows the core idea underneath
multi-agent frameworks.
Different roles own different
responsibilities.
Frameworks package these interaction
patterns.

**We already practiced role-based AI design with
planner, summarizer, and judge.**



Single Pipeline vs Multi-Agent Workflow

Single pipeline is best when

- task is clear
- workflow is predictable
- debugging matters
- latency / cost must stay low

Multi-agent helps when

- roles are naturally different
- critique / debate helps
- research is broad
- multiple perspectives improve quality

If the task is already linear, adding agents may add complexity without value.



Raw Python vs LangGraph vs MCP vs Multi-Agent

Raw Python

maximum control, manual orchestration

LangGraph

explicit state, nodes, edges, branching

MCP

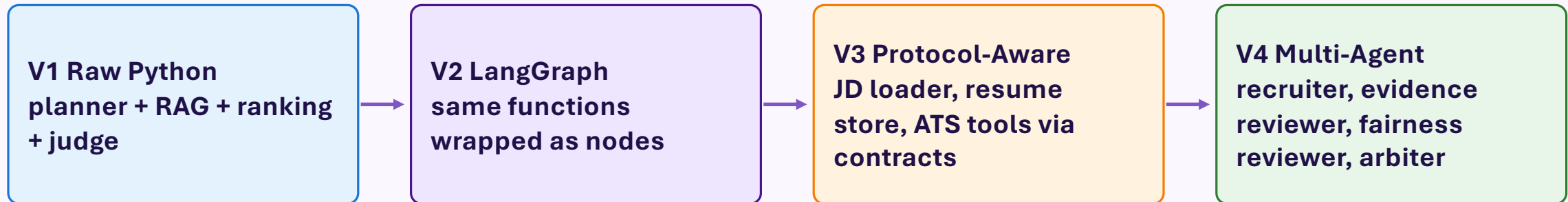
standardized tool/context access

AutoGen-style

role-based multi-agent coordination

**These are not all replacements for each other.
They live at different levels of the AI systems stack.**

Capstone Evolution: Resume Matching Copilot



**Same problem. Same core logic.
Increasingly mature coordination and integration layers.**

Final class message: learn the layers, then choose the tool.



What Students Should Build Next

Improve resume ingestion: messy PDFs, section-aware chunking, boilerplate cleanup.

Add evaluation dashboards: judge output, concerns, confidence, evidence links.

Add conditional judge: run it only when rankings are close or confidence is low.

Convert the raw Python workflow into LangGraph nodes.

Design MCP-style contracts for external tools and data sources.

Try a small multi-agent pattern: generator + critic or planner + executor.

**Best next capstone:
A production-aware
Resume Matching Copilot
with graph orchestration,
judge evaluation, and
explicit tool contracts.**

**Do not chase every
framework.
Build systems deliberately.**



Final Engineering Lessons

Prompts alone are not systems.
Structure makes AI output usable.
Retrieval makes answers grounded.
Agents need tools, state, and control.
Evaluation makes systems trustworthy.
Frameworks help only after you understand the architecture.
Protocols and multi-agent coordination matter when systems scale.

You now know how to think from raw prompts all the way to evaluated AI systems.

A good AI engineer understands where structure, retrieval, evaluation, and orchestration belong.



Final Class Q&A - Part 1

Use this as a live-answer cheat sheet during the final Week 6 discussion.

Getting Started + Core Architecture

How to choose the first AI project?

Pick clear input/output, repetitive manual reasoning, real value, and low risk. Start with internal copilots before high-stakes automation.

Start with the clearest workflow problem.

When to use frameworks?

Use them when orchestration, reuse, state, and team consistency get messy. Stay in plain Python while the system is still clean.

Frameworks should remove pain, not mystery.

How to improve retrieval quality?

Improve ingestion first: clean PDFs, better chunking, overlap, metadata, thresholds, top-k tuning, and reranking.

Bad retrieval is often an ingestion problem.

How much math is necessary?

Enough to understand embeddings, similarity, tokens, chunking, and uncertainty. You do not need to re-derive transformers.

Understand behavior, not research math.

One model or many?

Use one model for simplicity. Use multiple models only when different steps need different strengths: planning, judging, or final reasoning.

Use many models only when the workflow earns it.

Production evaluation?

Layer it: schema validation, test sets, judge/evaluator layers, logs, tracing, human spot checks, and fallback rules.

Important AI outputs should be validated or reviewed.



Final Class Q&A — Part 2

Advanced systems: judge layers, MCP, multi-agent choices, and final architecture philosophy.

Advanced AI Systems Choices

Role of judge layers?

Judge layers check grounding, fairness, confidence, and recommendation quality. They reduce blind trust in generated outputs.

Was my own output actually justified?

Where MCP may fit?

MCP helps when many tools, files, databases, and context providers need standardized access. It is interface design.

MCP makes systems easier to connect.

Is multi-agent worth it?

Worth it when specialized roles add value: planner, researcher, critic, judge, arbiter. Not worth it for simple linear workflows.

More agents is not the goal. Better coordination is.

How to compare with market agentic AI?

Our system has planner, retrieval, structured outputs, judge, and graph orchestration. Market systems add durability, observability, permissions, and deployment.

The gap is production hardening, not architecture basics.

Best final architecture answer?

There is no universal best stack. Choose based on task, risk, budget, latency, data quality, and reliability requirements.

Best architecture depends on the problem.

Best Closing Line

“You now know how to think from raw prompts all the way to evaluated AI systems.”

End the program with confidence and a systems-thinking frame.